

 Memory updated

Absolutely 👍 **This is now the proper 6.x roadmap we'll follow before moving to the next major topic.** No more surprise sub-sub-sub-lessons 😂



Next.js 6.x — RSC, Navigation & Caching

#	Topic	Status
6.11	RSC Payload + React Flight	🟡 In progress
6.12	Serialization, Client References & Server References	⌚
6.13	Server ↔ Client Component boundaries	⌚
6.14	<code>router.refresh()</code> vs navigation vs full reload	⌚
6.15	Router Cache + RSC Payload + navigation caching	⌚
6.16	DevTools experiment — inspect HTML, RSC requests, JS & Router Cache	⌚
6.17	Complete rendering/caching mental model + interview questions	⌚

Where we are right now

We've already covered quite a bit of **6.11**:

- React Flight = protocol/mechanism
- RSC Payload = Flight-formatted data
- Serialized representation
- Flight records/chunks
- Client References
- Server References
- How Client JS relates to RSC Payload
- How RSC connects to Router Cache and navigation

But we'll **finish 6.11 properly**, then move sequentially:

```
6.11  RSC + Flight
      ↓
6.12  Serialization + References
      ↓
6.13  Server/Client boundaries
      ↓
6.14  refresh vs navigation vs reload
      ↓
```



6.15 Router Cache + RSC + navigation
↓
6.16 🔍 REAL DevTools experiment
↓
6.17 🧠 COMPLETE mental model + interview prep
↓
🏁 6.x COMPLETE
↓
NEXT MAJOR TOPIC

And **6.16** is intentionally near the end because by then you'll understand *what you're looking at* when we open DevTools, instead of just staring at a terrifying wall of RSC gibberish. 😂

One clarification

The things we previously called **6.11.1 / 6.11.2 / 6.11.3** were essentially the deeper pieces of **6.11–6.13**. From now on, I'll use **your 6.11 → 6.17 numbering** as the canonical roadmap.

So next: **6.11 — continue/finish RSC Payload + React Flight**, and then **6.12**. 🚀



okay after completing 6.11.x series we open new chat for 6.12

📖 Memory updated

Yep 😊 **exactly.**

We'll finish the entire **6.11.x series in this chat:**

6.11
├─ 6.11.1
├─ 6.11.2
├─ 6.11.3
├─ ...
└─ 6.11.x
↓
🏁 COMPLETE



Then **stop here.**

You'll open a **new chat** and say:

"Continue Next.js 6.12"

And we'll start fresh with:

🚀 6.12 — Serialization, Client References & Server References

I'll treat your **6.11 → 6.17 roadmap** as the canonical roadmap going forward. 🍌

[cont...next](#)

🚀 6.11.4 — Inspecting an Actual RSC Request

Yep, continuing! 😊

Now we're moving from **theory** → **what actually happens on the network**.

1. Our experiment

We'll eventually create a small Next.js app with:

```
app/  
├─ layout.tsx  
├─ page.tsx  
└─ products/  
    └─ [id]/  
        └─ page.tsx
```



And something like:

```
/products  
  ↓  
click <Link>  
  ↓  
/products/123
```



Then we'll open:

Chrome → DevTools → Network

and watch what happens.

2. First establish the baseline

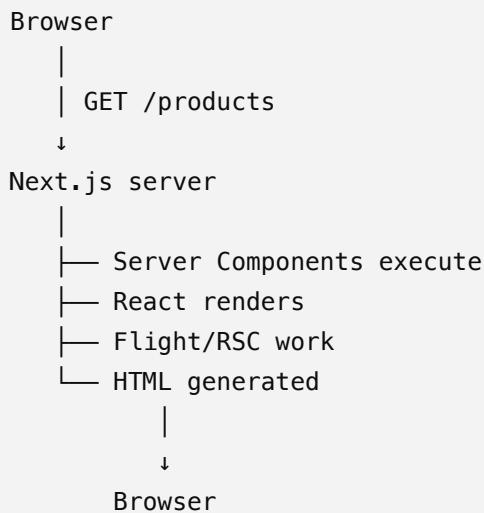
When you type:

```
http://localhost:3000/products
```



into the address bar, that's a **document navigation**.

Conceptually:



The browser receives an HTML document.

So Network will have a request whose response is HTML.

3. Now the important experiment

On `/products`, put:

```
</> TypeScript

import Link from "next/link";

export default function ProductsPage() {
  return (
    <div>
      <h1>Products</h1>

      <Link href="/products/123">
        Product 123
      </Link>
    </div>
  );
}
```



Now:

Don't refresh.

Instead:

click the Link.

This is where App Router's client navigation machinery becomes visible.

4. What changes?

Instead of:

```
click
↓
GET entire HTML document
↓
replace browser document
```



the App Router can do:

```
click
↓
Next.js client router
↓
request RSC/Flight data
↓
server renders destination Server Components
↓
RSC Payload
↓
browser
↓
React updates tree
```



The important thing is:

The browser doesn't need to replace the entire document just because the route changed.

5. What should you look for in Network?

Depending on your exact Next.js version and development/production mode, the request won't necessarily look identical to screenshots or tutorials online.

But look for evidence of an **RSC request**, including things such as:

RSC



and Next.js router-related headers/parameters.

You may see things along the lines of:

RSC: 1



and router state information such as:

Next-Router-State-Tree



The exact request details are implementation/version dependent.

That's why I don't want you memorizing a particular header as "the RSC request."

Instead, understand **why the request exists**.

6. What's actually coming back?

The response is not simply:

 HTML



```
<html>
...
</html>
```

Instead, the server can return the **Flight/RSC representation** for the requested route.

Conceptually:

RSC Response



```
record
record
record
record
record
...
```

Those records contain information React needs to update the Server Component tree.

7. Why does the response look so weird? 🤔

You might see something that resembles:

```
1:I[...]  
2:"..."  
3:["$","div",...]  
...
```



or other strange encoded records.

Don't panic.

That's exactly what we were talking about earlier.

It's not:

```
normal HTML
```



and it's not simply:

```
JSON
```



It's Flight's representation.

Think:

```
React tree  
  ↓  
Flight serialization  
  ↓  
RSC payload  
  ↓  
weird-looking network data
```



8. Now compare initial load vs client navigation

This is the important experiment.

Initial load

```
Address bar  
  ↓  
GET /products  
  ↓
```



```
HTML document
+
RSC-related data
+
client JS
↓
Browser
```

Client navigation

```
<Link>
↓
client router
↓
RSC request
↓
Flight/RSC response
↓
React
↓
DOM update
```



So the network behavior is different.

9. Here's where your earlier question gets answered

You asked previously:

"If the destination page has a completely different HTML structure, doesn't HTML need to be fetched?"

The answer is:

The DOM structure can completely change.

For example:

Current:

```
HTML
<main>
  <h1>Products</h1>
  <ul>...</ul>
</main>
```

</>



Destination:

 HTML

</>



```
<main>
  <h1>Product 123</h1>
  <section>...</section>
  <button>Buy</button>
</main>
```

The browser's DOM absolutely changes.

But React doesn't need a **new HTML document** to accomplish that.

It can receive the new RSC result:

```
RSC payload
  ↓
React reconciles
  ↓
DOM changes
```



DOM changes $\neq$ full document replacement.

That's the distinction.

10. Now bring Router Cache into the experiment

Suppose you click:

```
/products
  ↓
/products/123
```



and the RSC response arrives.

Conceptually:

```
RSC Payload
  ↓
Router Cache
  ↓
React
  ↓
DOM
```



The Router Cache can retain the RSC information associated with route segments.

So later navigation can potentially reuse information that was already prefetched/fetched.

11. And `<Link>` makes this even more interesting

If Next.js prefetches a route:

```
<Link href="/products/123">
```



the destination's RSC data can potentially be requested **before you click**.

Conceptually:

```
User sees Link
      ↓
prefetch
      ↓
RSC payload
      ↓
Router Cache
      ↓
User clicks
      ↓
reuse cached RSC data
```



That's one of the reasons App Router navigation can feel very fast.

12. One VERY important thing

Don't conclude:

"Router Cache stores the raw network response exactly as I see it."

That's too simplistic.

The Router Cache is an internal client-side cache of the **RSC-rendered route/segment information** used by the App Router.

The exact internal representation isn't something you should depend on as a public API.

So our mental model is:

```
Network
```

```
↓
```



```
Flight/RSC data
  ↓
Next.js client router
  ↓
Router Cache
  ↓
React
```

rather than:

```
Network response
  ↓
some visible text file
  ↓
Router Cache
```



13. One more DevTools experiment we'll do later

When we actually build the app, we'll compare these **three** things side-by-side:

A. Document request

```
GET /products
```



Look at:

```
Response → HTML
```



B. Client navigation request

```
<Link href="/products/123">
```



Look at:

```
Request headers
Response
RSC indicators
Flight data
```



C. Client JavaScript

Find the JS chunks and identify that:

Client Component



client JS bundle



This will visually connect:

HTML

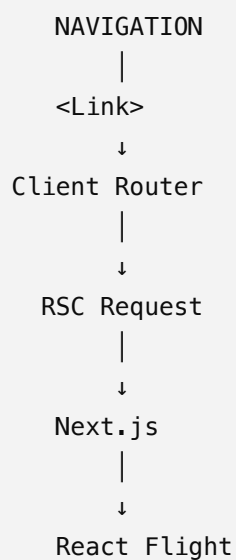
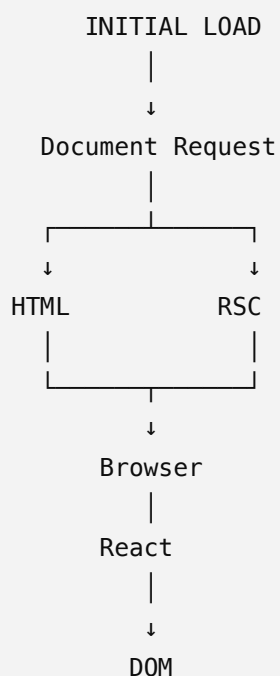
RSC Payload

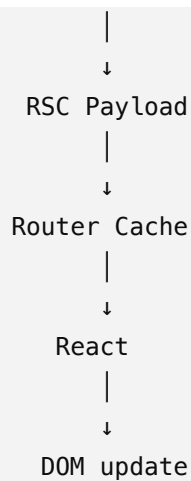
Client JS



which we've been discussing for the last several lessons.

The complete 6.11.4 mental model





And 6.11.4 is now conceptually complete. ✅

Next in our fixed roadmap:

6.11.5 — RSC Payload + Router Cache + Navigation Caching

Here we'll go deeper into **exactly what happens when the Router Cache is a HIT vs MISS**, including how prefetched RSC data gets reused.



😊 Perfect. Next we'll do **6.11.7 — the complete RSC + React Flight recap and interview questions.**

After that, **6.11.x is officially done 🏁**, and we'll start **6.12 in a new chat.**



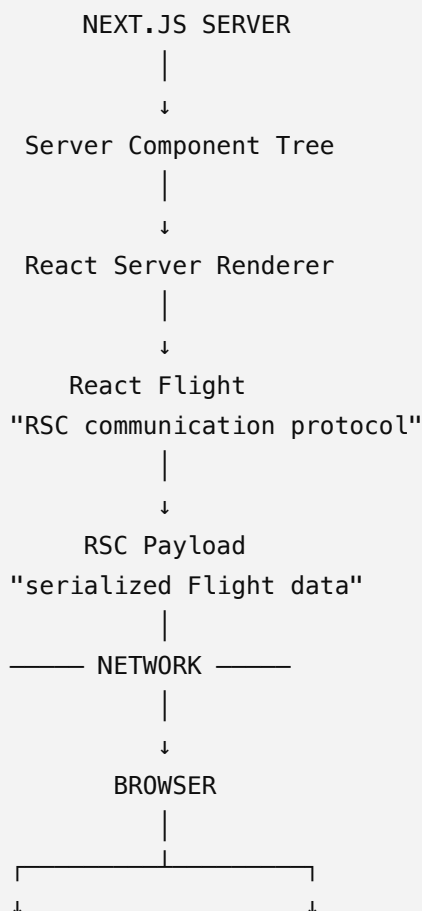
sure go ahead

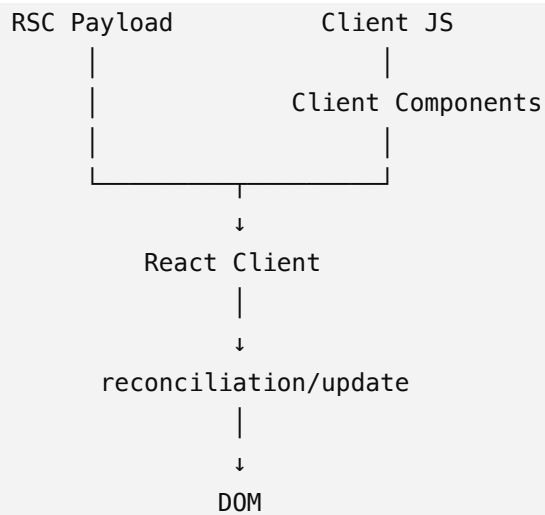
🏁 6.11.7 — Complete RSC Payload + React Flight Recap

Alright 😊 this is the **final 6.11.x lesson**. After this, we're done with RSC/Flight fundamentals and **6.12 will start in a fresh chat**, as planned.

Let's consolidate everything into one mental model first.

1. The entire RSC story in one picture





The biggest thing to remember:

Server Components don't need to be shipped to the browser as JavaScript. Their rendered result is represented in the RSC payload.

Client Components are different because their JavaScript needs to be available in the browser for interactivity.

2. Flight vs RSC Payload

This was one of your original questions.

React Flight

Think:

The protocol/mechanism used by React to transport Server Component information.

RSC Payload

Think:

The actual serialized data produced using that mechanism for a particular render/navigation.

So:

Flight
↓
protocol / mechanism

RSC Payload



↓
actual Flight data

Don't say:

"Flight is the payload."



Say:

"The RSC payload is transmitted using React Flight."



3. What does "serialized representation" mean?

Suppose the server has:

</> TypeScript

```
<Product  
  name="Dosa"  
  price={80}  
/>
```



React has an internal representation of this component tree.

That internal representation can't simply be sent directly over HTTP.

So React converts the relevant information into a **transferable Flight representation**.

React tree

↓
serialization
↓
Flight representation
↓
network
↓
client React



So:

Serialized representation = a network-transferable representation of the Server Component tree, supported values, and relevant references.

And importantly:

```
Flight serialization  
≠  
JSON.stringify()
```



Flight has React-specific semantics.

4. Client References

Suppose:

```
</> TypeScript  
  
'use client';  
  
export default function AddButton() {  
  return <button>Add</button>;  
}
```



The Server Component tree can contain:

```
</> TypeScript  
  
<AddButton />
```



The server doesn't need to send the component's source code inside the RSC payload.

Instead, Flight can represent a **Client Reference**.

Conceptually:

```
Server  
  |  
  | "Client Component X goes here"  
  ↓  
Client Reference  
  |  
  ↓  
Browser  
  |  
  ↓  
client JS module  
  |  
  ↓  
AddButton
```



So:

Client Reference

≠

Client Component source code



It's a reference to the client-side module/export.

5. Client JS vs RSC Payload

Don't mix these up.

BROWSER



RSC data
tree/results
references

Client JS
component code
event handlers

The two work together.

RSC Payload

Answers roughly:

"What Server Component result/tree should React use?"

Client JS

Provides:

"Here is the executable code for Client Components."

6. Server Functions / Server References

Now reverse the direction.

```
</> TypeScript

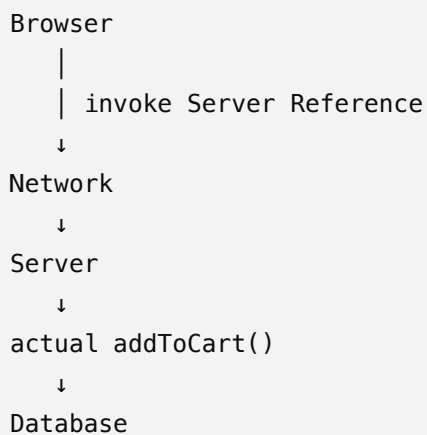
'use server';

export async function addToCart(productId) {
  await db.cart.add(productId);
}
```

The browser doesn't receive the implementation.

Instead, it can receive/use a **Server Reference**.

Conceptually:



And remember the important distinction:

```
function that happens to execute on server
    ≠
Server Function
```

A Server Function has explicit Server Function semantics, typically via:

```
</> TypeScript

'use server';
```

7. Ordinary function vs Server Function

Ordinary server-side function

```
</> TypeScript

function getProducts() {
  return db.products.findMany();
}
```

```
}
```

It may execute on the server.

But it isn't automatically remotely invokable.

server-side function

↓

runs on server

↓

ordinary function



Server Function

`</> TypeScript`



```
'use server';
```

```
export async function addToCart() {  
  ...  
}
```

Server Function

↓

Server Reference

↓

can be invoked through supported server-action mechanisms



8. Serialization boundary

Suppose:

`</> TypeScript`



```
export default function Page() {  
  const product = {  
    id: 1,  
    name: "Dosa"  
  };  
  
  return <ProductCard product={product} />;  
}
```

and:

```
</> TypeScript

'use client';

function ProductCard({ product }) {
  ...
}
```

The `product` data crosses:

```
Server
  |
  | serializable value
  ↓
Client boundary
  |
  ↓
Client Component
```

But an ordinary server function cannot simply be passed the same way:

```
</> TypeScript

<ProductCard
  onSave={someServerFunction}
/>
```

unless that function has the appropriate **Server Function/reference semantics**.

9. 'use client' vs 'use server'

Keep this distinction very clear.

'use client'

```
'use client'
  ↓
Client Component boundary
  ↓
client module graph
  ↓
browser JS
```

'use server'

```
'use server'
```

↓

Server Function semantics

↓

Server Reference

↓

remote invocation



And:

'use client' does **not** simply mean "this component only renders in the browser."

That's a common oversimplification.

10. RSC navigation

Now suppose:

```
/products
```



and:

```
</> TypeScript
```

```
<Link href="/products/123">
```



is clicked.

The important flow is:

```
<Link>
```

↓

Next.js Client Router

↓

RSC request

↓

Server Components execute/render

↓

React Flight

↓

RSC Payload

↓

Browser

↓

React reconciliation



↓
DOM update

Notice:

No full HTML document replacement is required for this client-side navigation.

11. DOM can change without replacing the document

This was another question you asked earlier.

Suppose:

Before

```
<main>
  <h1>Products</h1>
  <ProductList />
</main>
```



After navigation

```
<main>
  <h1>Product 123</h1>
  <ProductDetails />
</main>
```



The DOM can change substantially.

But:

```
DOM changed
  ≠
whole HTML document replaced
```



React can update the existing DOM based on the new RSC result.

That's a **very important App Router concept**.

12. Router Cache

The RSC payload received during navigation can be used by the App Router and stored in its client-side Router Cache.

Conceptually:

Server



RSC Payload



Router Cache



React



DOM



Router Cache is:

Client-side cached RSC route/segment information used by the App Router.

Don't confuse it with:

Data Cache

Full Route Cache

browser HTTP cache

JS bundle cache



They're different things.

13. Cache HIT vs MISS

HIT

Navigation



Router Cache



HIT 



reuse cached RSC information



React



DOM



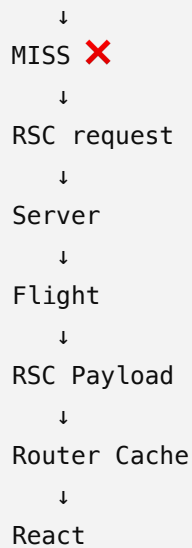
MISS

Navigation



Router Cache



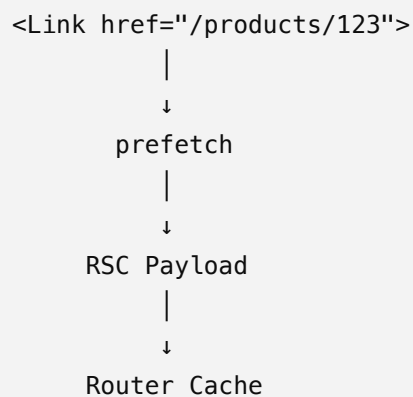


This is the core navigation-caching model.

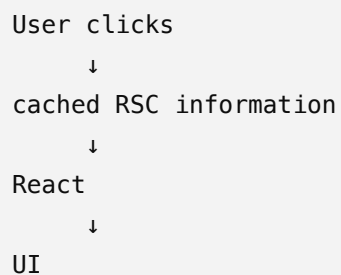
14. Prefetching

A `<Link>` can also be prefetched in applicable situations.

Conceptually:



Then:



So a request can happen **before the user actually navigates**.

15. Streaming + Suspense

Now connect the last part.

Suppose:

```
Header      → fast
Products    → fast
Reviews     → slow
```



With:

```
</> TypeScript

<Suspense fallback={<ReviewsSkeleton />}>
  <Reviews />
</Suspense>
```



React can conceptually produce:

```
Header
  ↓
Products
  ↓
ReviewsSkeleton
```



then later:

```
Reviews
  ↓
replace fallback
```



Flight allows the RSC result to be **streamed progressively**.

So:

```
Suspense
  ↓
defines boundary

Streaming
  ↓
delivers results progressively

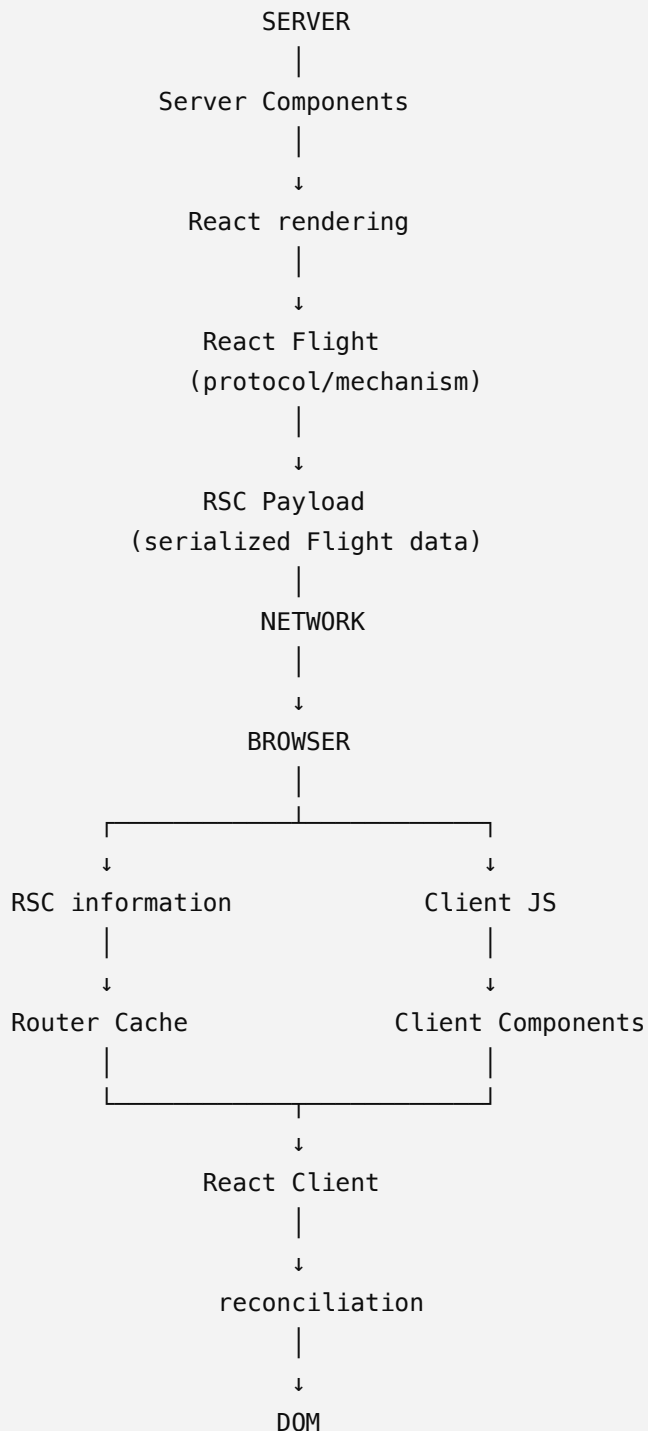
Flight
  ↓
transports RSC representation
```



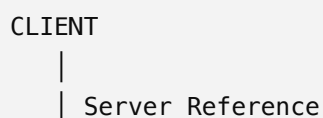
These aren't interchangeable concepts.

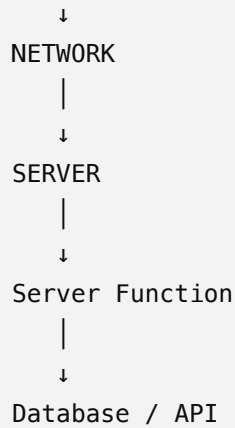
16. The complete architecture 🔥

Here's your final **6.11 mental model**:



For server mutations:





🎯 17. Interview Questions

Now let's test whether you've actually got it rather than just nodding along. 😂

Q1. What is React Flight?

Answer:

React Flight is the protocol/mechanism React uses to communicate the Server Component tree between server and client.

Q2. What is the RSC Payload?

The RSC Payload is the serialized Flight data representing the Server Component result/tree, supported values, and relevant references for a particular render or navigation.

Q3. Is RSC Payload HTML?

No.

HTML

→ document/UI representation

RSC Payload

→ React Server Component representation



Q4. Is RSC Payload JSON?

Not simply.

Flight has its own serialization format and semantics beyond ordinary JSON serialization.

Q5. Does the browser receive Server Component source code?

No.

The Server Component executes on the server, and its result is represented in the RSC payload.

Q6. Does the browser receive Client Component JavaScript?

Yes.

Client Components need client-side JavaScript for their interactivity.

Q7. What is a Client Reference?

A reference in the RSC system to a Client Component/module that the browser can resolve to its client-side code.

Q8. What is a Server Reference?

A reference to a Server Function that allows the client to request its invocation without receiving the function implementation.

Q9. Can any function become a Server Function just because it runs on the server?

No.

runs on server
≠
Server Function



Server Function semantics are explicitly established, typically with `'use server'`.

Q10. During `<Link>` navigation, does Next.js normally download a complete new HTML document?

No.

For client-side App Router navigation, Next.js can request the RSC payload and use it to update the existing React tree and DOM.

Q11. Can the DOM structure change during client navigation?

Absolutely.

```
existing DOM
  ↓
React receives new RSC result
  ↓
reconciliation
  ↓
new DOM structure
```



But the entire browser document doesn't have to be replaced.

Q12. What does Router Cache store?

Client-side RSC route/segment information used by the App Router for navigation and reuse.



18. Your 30-second interview answer

If they ask:

"Explain React Server Components and RSC Payload in Next.js."

You can say:

"Next.js uses React Server Components to execute server-side component logic. React's Flight protocol provides the mechanism for representing and transporting the Server Component result to the client as an RSC payload. The payload contains serialized component-tree information, supported values, and references to Client Components. The client combines that information with the Client Component JavaScript and React uses it to construct or reconcile the UI."

During App Router navigation, the RSC payload can be fetched and stored in the client Router Cache, allowing the existing application tree to be updated without a full document reload."

That's a very solid mid-level answer. 🎯

🚩 6.11.x — COMPLETE

You now have:

6.11 RSC Payload + React Flight



- |
- |— Flight
- |— RSC Payload
- |— serialization
- |— Client References
- |— Server References
- |— Server Functions
- |— RSC navigation
- |— Router Cache relationship
- |— prefetching
- |— streaming
- |— Suspense

🚩 DONE

And now we STOP this chat. 🙅

As planned, don't start 6.12 here.

Open a new chat and say:

"Continue Next.js 6.12"

Then we'll begin the next major piece:

🚀 6.12 — Serialization, Client References & Server References

We'll go deeper there, without mixing it back into 6.11. 😊



server function vs server side function invocation and diff

Yes — this distinction is **super important**, because "server-side function" and "Server Function" sound almost identical. 😊

1. Server-side function

A **server-side function** simply means:

A normal function whose code executes on the server.

Example:

`</>` TypeScript

```
async function getProducts() {  
  const products = await db.products.findMany();  
  return products;  
}  
  
export default async function Page() {  
  const products = await getProducts();  
  
  return <ProductList products={products} />;  
}
```

Here:

```
Page()  
  ↓  
getProducts()  
  ↓  
Database
```

`getProducts()` runs on the server.

But it's just a **normal function call**:

```
Page()  
  |  
  | normal JS call  
  ↓  
getProducts()
```

There is **no network request** between `Page` and `getProducts`.

The function isn't remotely callable by the browser.

2. Server Function

A **Server Function** is different.

It is explicitly given Server Function semantics, typically with:

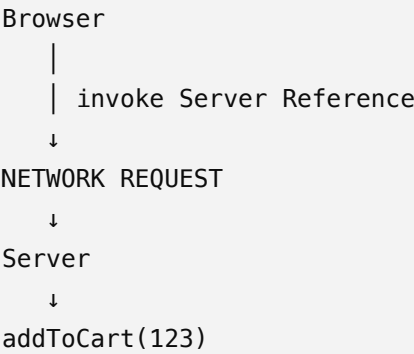
`</> TypeScript`

`'use server';

export async function addToCart(productId) {
 await db.cart.add(productId);
}`

Now the function can be exposed through a **Server Reference**.

Conceptually:



So there's a fundamental difference:

	Server-side function	Server Function
Runs on server	✓	✓
Normal JS function	✓	Also a function, but special semantics
'use server'	✗	✓
Callable remotely from client	✗	✓ through supported mechanisms
Server Reference	✗	✓
Network involved when called from server code	✗	Not necessarily

	Server-side function	Server Function
Designed for client → server invocation	✗	✓

3. "Server-side function invocation" means something different

Suppose:

`</>` TypeScript

```
async function getUser() {  
  return db.user.find();  
}  
  
async function Page() {  
  const user = await getUser();  
}
```



The invocation is:

```
Page()  
|  
| function call  
↓  
getUser()
```



Everything is happening inside the **same server execution environment**.

There's no:

```
browser  
↓  
network  
↓  
server
```



It's simply:

```
Server  
|  
├─ Page()  
|   |  
|   └─ getUser()
```



|
└─ DB

4. Server Function invocation is different

Now:

```
</> TypeScript   
  
'use server';  
  
export async function addToCart(productId) {  
  await db.cart.add(productId);  
}
```

And the browser wants to invoke it.

The browser doesn't have:

```
</> JavaScript   
  
addToCart() // implementation
```

Instead it has a reference.

Conceptually:

```
Client   
|  
| Server Reference: XYZ  
| arguments: 123  
↓  
Network  
|  
↓  
Server  
|  
| resolve XYZ  
↓  
addToCart(123)
```

That's a **remote invocation**.

5. Here's the easiest way to remember it 🧠

Ask:

"Who is calling the function?"

If it's:

Server code



Server function



that's just a normal function invocation, even if the function is a Server Function.

No special network trip is needed.

But if:

Browser



Server Function



then the browser can't directly execute it.

It invokes the **Server Reference**, which causes communication with the server.

6. This is the subtle point

A Server Function doesn't mean:

"Every time this function is called, a network request happens."



It means:

The function has semantics that allow it to be referenced and invoked from the client.

So:

</> TypeScript

```
'use server';
```

```
export async function addToCart(id) {  
  ...  
}
```



If server code does:

```
</> TypeScript  
  
await addToCart(123);
```



that's simply:

```
Server  
  ↓  
normal function invocation  
  ↓  
addToCart()
```



But when the **client** invokes it through its **Server Reference**, then:

```
Client  
  ↓  
Server Reference  
  ↓  
network  
  ↓  
Server Function
```



7. One more example

Normal server-side helper

```
</> TypeScript  
  
async function calculatePrice(productId) {  
  const product = await db.products.find(productId);  
  return product.price * 1.18;  
}
```



Used by:

```
</> TypeScript  
  
export default async function Page() {  
  const price = await calculatePrice(123);  
  
  return <div>{price}</div>;  
}
```



Flow:

```
Page()  
↓  
calculatePrice()  
↓  
DB
```



No client involvement.

Server Function

```
</> TypeScript  
  
'use server';  
  
export async function purchase(productId) {  
  await db.orders.create({ productId });  
}
```



Client:

```
</> TypeScript  
  
'use client';  
  
import { purchase } from './actions';  
  
export function BuyButton({ productId }) {  
  return (  
    <button onClick={() => purchase(productId)}>  
      Buy  
    </button>  
  );  
}
```



Conceptually:

```
CLICK  
↓  
purchase(123)  
↓  
Server Reference  
↓  
NETWORK  
↓  
Server
```



↓
actual purchase(123)
↓
DB

The interview distinction

If they ask:

"What's the difference between a server-side function and a Server Function?"

Say:

"A server-side function is simply a normal function that executes in the server environment. It can be called by other server code through an ordinary JavaScript function call. A Server Function has explicit Server Function semantics, typically through `'use server'`, which allows it to be represented by a Server Reference and invoked from client code through the server-action mechanism. The function implementation remains on the server."

And the one-liner:

Server-side function
= where it executes



Server Function
= special semantics for server invocation from the client

 That distinction is exactly what I want you to carry into 6.12.



ChatGPT can make mistakes. Check important info.